

A Study on Predicting Software Defects with Machine Learning Algorithms

Anjali C, Julia Punitha Malar Dhas*, J. Amar Pratap Singh

Dept. of CSE, Noorul Islam Centre for Higher Education, Kumaracoil, India,

*Dept. of CSE, Karunya Institute of Technology and Sciences, Coimbatore, India

Email: anjali.c@mbcet.ac.in juliapunitha@karunya.edu* japsindia@yahoo.com

Abstract—Software Defect Prediction (SDP), even in its early stages, is a crucial and significant activity. SDP has recently received a lot of attention as a quality assurance method. Massive amounts of reports and defect data may be generated by the component services. Although much emphasis has been placed on developing defect prediction models using machine learning (ML), some work has been done to determine how effective source code is. ML is a supervised algorithm that is used to produce better results. To appreciate defect prediction in SOS better, this paper suggests a fault diagnosis framework based on the web access to care.ML tools such as Random Forest (RF), Decision Tree (DT), and Support Vector Machine (SVM) are used to evaluate the model's utility, and certain metrics are also constructed using feature extraction techniques. Finally, the performances of the ML algorithms are compared and the better one is analyzed.

Index Terms—Decision tree, machine learning, Random Forests, Software defect prediction, Support Vector Machine

I. INTRODUCTION

From software formulation to blog software maintenance, software engineering (SE) is a skill that spans a broad range of software design [1].SDP is the most important and dynamic area of SE research, and it is critical to Software Quality Assurance (SQA) [2], [3]. Because of the increasing complexity and interdependence of software systems, it has become impossible to accomplish it with minimum effort, excellent quality, and manageability [4], [5]. An SD may occur if the client's requirements were not met by the earlier software product [6].SDS has the potential to reduce the quality of software while increasing development costs. SDP is important to ensure the components of a software application, result in acceptable development costs and quality recovery by identifying defect-prone instances before the experiment. It also includes partitioning the components into different types of systems that aid in the development of the validating process by focusing on testing and evaluating defective ones [7]. Defects have a negative impact on the reliability and quality of software [8].

This paper presents WSDL files generated from experimental data sets from benchmarked systems. The performance measures are calculated using a variety of ML techniques, including DT, RF, and SVM. Experiments are carried out to assess the effectiveness with the help of some performance measures.

The insufficient requirements, software developers could not be more qualified for software projects. One of a project

manager's shortcomings could be that they do not adhere to the table's requirements for the software development life cycle.

TABLE I
DEFECT PERCENTAGE

Software Development Phase	The defect may occur in a percentage
Requirement Phase	21%
Designing phase	25%
Coding Phase	35%
User Manual	11%
Bad Fixes	9%

- The following three core concerns in the project lifecycle tend to receive more focus from software efforts.

- Prediction of software defects using a vast amount of data [9].

- To ensure program dependability, estimate software time.

- Test software process and design will also have an impact on the density and amount of software flaws [10].

To create high-quality software, defect prediction is a crucial task. To satisfy users and enhance the software's overall performance, defect prediction comes before the software release.

II. RELATED WORKS

This section offers a brief summary of existing SDP methods. For SDP's initial phases, some researchers have used ML algorithms. Here, certain works specifically conflict with one another. Czibula et al [11].s model for SDP was developed using associations discovered through research. They concentrate on studying NASA data sets. The model performance comparison was done using the computed values. The research examined the DP-CNN in terms of F-measure in defect prediction on seven distinct datasets. Overall, the results show that the DP-CNN outperformed the current technique by an average of 14% [14].

The hybrid feature selection (HFS) strategy was proven by Jacob and Raju using NASA datasets. The performance of HFS is compared with ML algorithms. HFS surpasses testing data by improving the accuracy rate from 82% to 90%. Bashir et al. [12] presented a basis for implementing the SDP method that combines features such as sampling of data and filtering to counteract class imbalance, noise, and high dimensionality. In terms of overall experimental results,

the proposed work has given a better outcome. A method for SDP that classifies defective and non-defective objects using a hybridized gradual relational association and ANN has been proposed. As a result, many researchers have used machine learning techniques to predict disease in various healthcare datasets. Once they propose an optimal solution, they must understand the security of the system. To do so, software defects that may occur should be anticipated, resulting in a decrease in software system quality. These are the reasons for carrying out this research.

III. PROPOSED METHODOLOGY FOR DEFECT PREDICTION IN SOS

The most commonly used techniques are ML-based modeling methods. Fig. 1 shows the model and research approach for predicting software defects. The analysis and prediction process both employed the KC1 dataset. Using 10-fold cross-validation, the dataset was trained and tested. Different error measures and metrics are then used to evaluate the classifiers' output. Some analytical assessments are R2, mean absolute error, root means squared error, relative absolute error, and root relative squared error (RRSP). These metrics show the gap between the actual and planned amount of flaws in the dataset. Knowing E to be the sum of E's real flaws, E projected problems and E

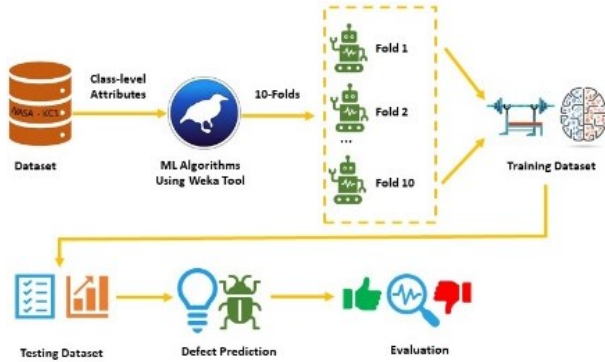


Fig. 1. Model Proposed Software Defect Prediction

Fig 2 depicts an overview of the machine learning-based categorization models used to predict software defects.

The first step in developing A fault predictive algorithm is to derive data instances from software archives as version control systems (like SVN, RF, and DT), issue tracking (like Bugzilla, Jira), and so on. The OS also keeps the data files and some solicit messages, and the issue radars store basic defect information. Depending on the forecasted level, each instance may be a code update, method, class, source code file, or package. The software archives are the source of several defect diagnosis metrics (features) hired in the case [12]. The metric values indicate how difficult the code is or how complex the SDP is.

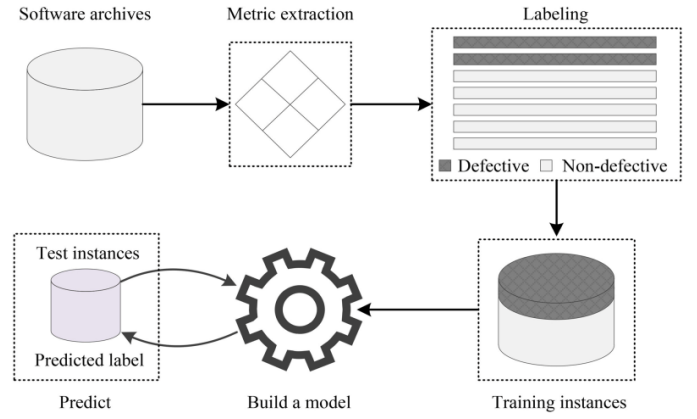


Fig. 2. Software Defect Prediction Process

IV. CLASSIFICATION TECHNIQUES

This section discusses classification models for predicting web service defects that have been investigated. Even though web service defect prediction is a supervised learning problem, the process of dividing classes is accomplished by examining the relevant metrics among both samples of known classes.

A. Support Vector Machine (SVM)

This section discusses classification models for predicting web service defects that have been investigated.

$$DF(b) = \sum_{i=0}^n b_i \propto_i K(a_i, a) + z \quad (1)$$

Where $(a_i, b_i), \dots, (a_n, b_n)$ are samples, $\alpha = (i = 1, 2, \dots, m)$, $K()$ denotes a kernel function and z is a bias. Input is classified as in equation (2)

$$a = \begin{cases} +1 & \text{if } DF(a) > 0 \\ -1 & \text{if } DF(a) < 0 \end{cases} \quad (2)$$

B. Decision Tree(DT)

The DT classifier is a machine learning technique used to predict categorical attributes or solve classification problems. In this study, metrics are dependent variables in a binary-valued output (defected or non-defected).

C. Random Forest(RF)

A supervised learning approach for regression and classification is the RF algorithm. Instead of employing a tree crown decision, it generates a forest of judgment trees and uses committee voting to determine the optimal answer [13].

RF is a supervised learning method that generates each tree by combining tree predictors. The strength of the individual trees in the forest and their correlation both affect the overfitting error of a tree classifier.

D. Performance measures

1) *Accuracy*: The division of correct predictions by the accurate helps to yield classification accuracy value. Equation (3) represents the similarity between the measured and the standard values.

$$accuracy = \frac{TP + TN}{TP + FN + FP + TN} \quad (3)$$

2) *Precision*: The measure of inconsistency that finds when repeatedly using the same instrument gives the precision value, defined as

$$precision = \frac{TP}{TP + FP} \quad (4)$$

3) *Recall*: Another important statistic is recalled, defined as the partition of input samples into classes that the model accurately predicts. A recall is calculated as follows

$$recall = \frac{TP}{TP + FN} \quad (5)$$

4) *F Score*: F Score is one of the commonly known values that can get by merging the precision and recall, defined as

$$F1Score = 2 * \frac{TP}{2 * TP + FP + FN} \quad (6)$$

V. EXPERIMENTAL RESULTS

This study's main objective is to determine how stable web service flaws are. Weka 3.8.3 console tests to spot code bugs. The examined dataset was exposed to percentage split testing (60, 70, 80, and 90%) and cross-validation (10-fold).

TABLE II
PERFORMANCE USING DECISION TREE

Measures	Dataset			
	OWLS-TC4	eBay603	eBay1055	eBay1061
Accuracy	97.77	98.65	98.11	98.85
F-score	0.91	0.92	0.92	0.93
Recall	0.93	0.83	0.79	0.79
Precision	0.91	0.92	0.93	0.89
AUC	0.83	0.91	0.92	0.89

In this study, the values obtained of validity, recall, specificity, f-measure, and AUC are used to evaluate the accuracy of web source code criteria for defect predictability, as shown in Table II. Tables II, III, and IV report the accuracy of different classifiers, including DL, RF, and SVM.

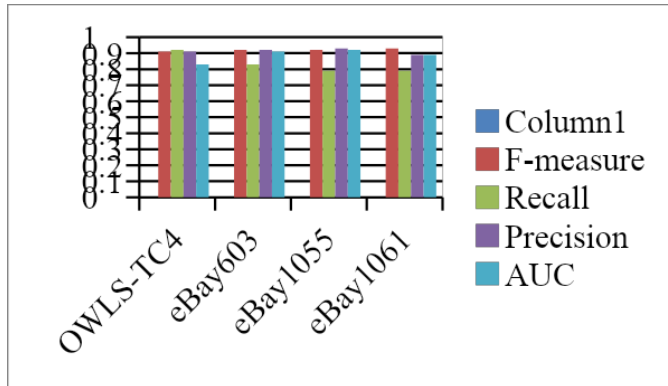


Fig. 3. Performance measures using the DT algorithm

Key parameters including precision, F-measure, recall, and AUC for the ML approach are vividly depicted in Fig 3, 4, and 5 for all datasets considered carefully.

TABLE III
PERFORMANCE USING RANDOM FOREST

Measures	Dataset			
	OWLS-TC4	eBay603	eBay1055	eBay1061
Accuracy	96.72	97.64	97.11	97.91
F-score	0.92	0.93	0.94	0.94
Recall	0.93	0.84	0.89	0.81
Precision	0.90	0.91	0.90	0.90
AUC	0.86	0.92	0.91	0.88

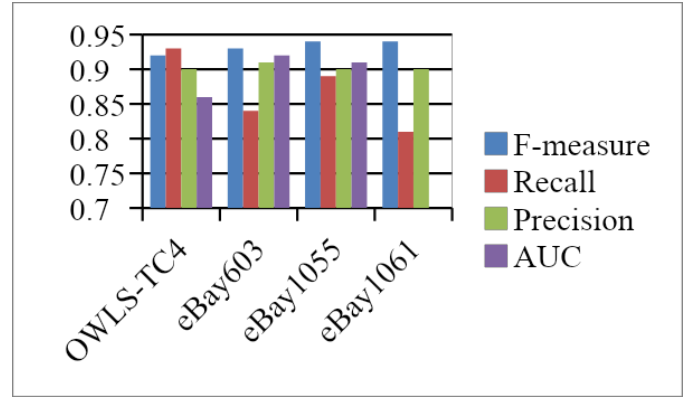


Fig. 4. Performance measures using the RF algorithm

TABLE IV
PERFORMANCE USING SVM

Measures	Datasets			
	OWLS-TC4	eBay603	eBay1055	eBay1061
Accuracy	94.67	96.64	96.15	97.88
F-score	0.90	0.91	0.91	0.92
Recall	0.91	0.82	0.78	0.78
Precision	0.90	0.91	0.92	0.87
AUC	0.82	0.90	0.91	0.88

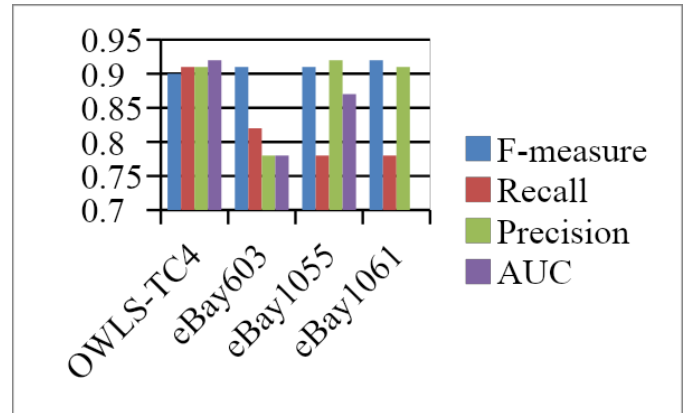


Fig. 5. Performance measures using the SVM algorithm

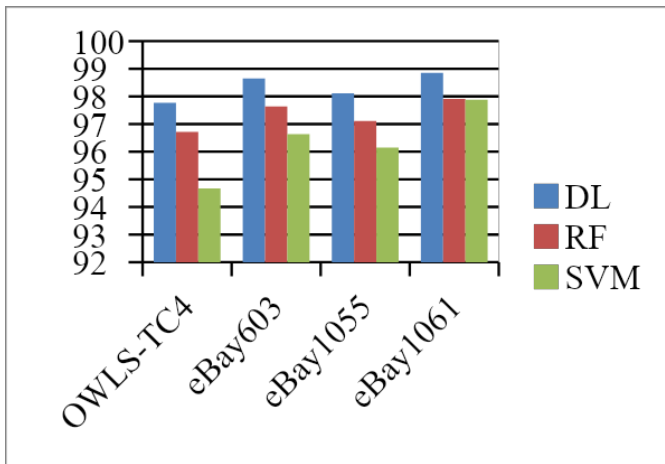


Fig. 6. Accuracy graph for ML algorithms

Fig 6 compares the accuracy of all algorithms tested on datasets such as OWLS-TC4, eBay603, eBay1055, and eBay1061.

Also, the DT algorithm beats all other machine learning algorithms, including RF and SVM.

The ideal category with the finest ability to anticipate software problems is tried in this experimental study, which uses seven machine learning algorithms to predict software defects before they're released into actual life or sold to consumers.

Any machine learning method can be evaluated in a variety of ways, and model evaluation is a crucial component of any project. In this experiment, the model performance is assessed using the assessment indicators listed above. First, it should be noted that RF and Decision tree algorithms outperform other learning algorithm categories in these experimental results, which are displayed in a graph. Despite using datasets of various sizes, no significant performance differences were found.

According to the findings of this experimental study, ensemble learners such as Random Forests and Decision Trees, which are tree-structured classifiers, perform better than their competitors at defect prediction. Defect prediction is particularly better at predicting software defects.

VI. CONCLUSION AND FUTURE WORK

The proposed work has been used to predict the defect tendency using source code metrics. The goal is to test the association between source code and the defect-proneness of web services. Six machine learning techniques were used to analyze data generated by three different versions of the eBay web services and OWLS-TC4 (DT, SVM, and RF). To exemplify the effect, a range of performance criteria can be used, including accuracy, f-measure, recall, and precision. Experiment results show that in the vast majority of the cases, DT outperforms other ML techniques in modeling web service defects. Data accessibility and transparency. Process metrics were utilized to make extremely accurate defect prediction

models using a variety of independent variables for recent SDP. Studies on defect prediction put more emphasis on practical performance.

Designing a more proactive strategy will be a future issue. Keeping Up with the Rapid Rate of Development. The release cycle of software is currently being shortened to days or even hours in the rapidly evolving corporate environment of today. With regard to effort, one idea is that rather than only enhancing precision and recall, we should test our prediction models in real-world contexts. Recent research has attempted to address these issues when taking effort into account. Accuracy in software engineering papers can be enhanced by creating replication packages that secure experiment validity and transparency and allow others to replicate their research and/or evaluate the efficacy of new designs with the originals using the same datasets.

REFERENCES

- [1] A. O. Balogun, A. O. Bajeh, V. A. Orie, and A. W. Yusuf-asaju, "Software defect prediction using ensemble learning: an ANP based evaluation method," *Journal of Engineering Technology*, vol. 3, no. 2, pp. 50–55, 2018.
- [2] Z. Li, X.-Y. Jing, and X. Zhu, "Progress on approaches to software defect prediction," *IET Software*, vol. 12, no. 3, pp. 161–175, 2018.
- [3] R. Malhotra and S. Kamal, "An empirical study to investigate oversampling methods for improving software defect prediction using imbalanced data," *Neurocomputing*, vol. 343, pp. 120–140, 2019.
- [4] D.-L. Miholca, G. Czibula, and I. G. Czibula, "A novel approach for software defect prediction through hybridizing gradual relational association rules with artificial neural networks," *Information Sciences*, vol. 441, pp. 152–170, 2018.
- [5] A. Alsaedi and M. Z. Khan, "Software defect prediction using supervised machine learning and ensemble techniques: a comparative study," *Journal of Software Engineering and Applications*, vol. 12, no. 05, pp. 85–100, 2019.
- [6] J. Li, P. He, J. Zhu, and M. R. Lyu, "Software defect prediction via convolutional neural network," in *Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability & Security. QRS*, 2017, pp. 318–328.
- [7] S. Jacob and G. Raju, "Software defect prediction in large space systems through hybrid feature selection and classification," *International Arab Journal of Information Technology*, vol. 14, no. 2, pp. 208–214, 2017.
- [8] D.-L. Miholca, G. Czibula, and I. G. Czibula, "A novel approach for software defect prediction through hybridizing gradual relational association rules with artificial neural networks," *Information Sciences*, vol. 441, pp. 152–170, 2018.
- [9] G. P. Bhandari and R. Gupta, "Machine learning based software fault prediction utilizing source code metrics," *2018 IEEE 3rd International Conference on Computing, Communication and Security (ICCCS)*, pp. 40–45, 2018.
- [10] A. Kumar, D. Prusti, I. S. Purusottam, and S. K. Rath, "Real time SOA based credit card fraud detection system using machine learning techniques," *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–6, 2021.
- [11] N. Li, M. Shepperd, and Y. Guo, "A systematic review of unsupervised prediction techniques for software defect prediction," *Information and Software Technology*, vol. 122, pp. 1–12, 2020.
- [12] S. Wang, T. Liu, J. Nam, and L. Tan, "Deep Semantic Feature Learning for Software Defect Prediction," *IEEE Transactions on Software Engineering*, vol. 46, pp. 1267–1293, 2020.
- [13] H. K. Dam et al., "Lessons Learned from Using a Deep Tree-Based Model for Software Defect Prediction in Practice," *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pp. 46–57, 2019.